



PROGRAMOWANIE URZĄDZEŃ MOBILNYCH 2

WYKŁAD 1

- Organizacja kursu
- Nawigacja w Jetpack Compose
- Typowane trasy, menu i grafy

Rafał Lewandków

pokój 075

rafal.lewandkow2@uwr.edu.pl

Wykład: 15 godz. Laboratorium: 30 godz.

Materiały: <https://github.com/RafLew84/ProgUM>

Literatura obowiązkowa i zalecana

Dokumentacja: kotlinlang.org/docs/home.html

Kursy: developer.android.com/courses

6 list zadań

Każda lista zawiera zadania i pulę pytań.
Z każdej listy student otrzymuje osobną ocenę.

Jedna lista może pozostać niezaliczona

Za nieoddaną lub niezaliczoną listę wpisuje się 2,0.

Średnia ze wszystkich sześciu ocen

Do uzyskania oceny 3,0 potrzebna jest średnia
co najmniej 3,0.

Listy oddaje się podczas zajęć laboratoryjnych.

Zadania + odpowiedzi = maksymalnie 10 pkt

Punktacja	Przykład 1	Przykład 2	Przykład 3
Zadania	5	6	7
Pytania (po 1 pkt)	5	4	3
Razem	10	10	10

Za odpowiedź: 0 albo 1 pkt.

Bez punktów częściowych za pytania.

Liczba pytań zależy od maksimum za zadania,
a nie od wyniku uzyskanego przez studenta.

Punkty	Ocena
poniżej 6	2,0
7	3,0
8	3,5
9	4,0
10	4,5
10	5,0

Przykład

Zadania: 4 z 6 pkt

Pytania: 3 z 4 pkt

Razem: 7/10 pkt

Ocena: 3,5

Student losuje 4 pytania, choć za zadania otrzymał 4 pkt.

Termin oddania

Każda lista ma własny termin podany w jej treści.

Opóźnienie

Za każdy tydzień opóźnienia ocena z listy jest obniżana o 1,0.

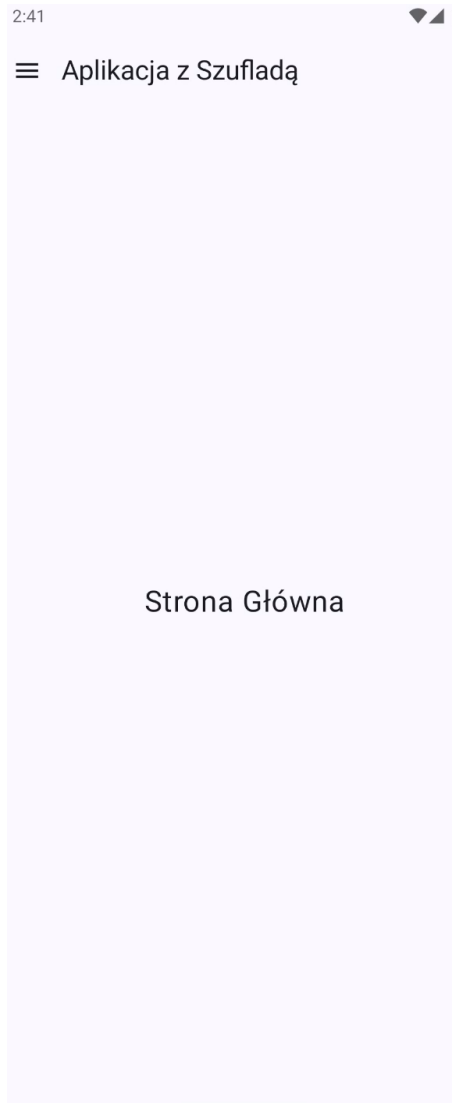
Obecność

Dopuszczalne są trzy nieobecności nieusprawiedliwione na laboratorium.

- 1** Organizacja kursu i nawigacja Compose
- 2** Korutyny i wątek główny
- 3** Współbieżność, równoległość i asynchroniczność
- 4** Architektura aplikacji: MVC, MVP, MVVM
- 5** Flow, StateFlow i SharedFlow
- 6** withContext, flowOn, combine, stateIn, shareIn
- 7** Kanały i komunikacja między korutinami

- 8** Ustawienia: SharedPreferences i DataStore
- 9** SQLite i Room: encje, DAO, CRUD
- 10** Komunikacja z API: Retrofit
- 11** Wstrzykiwanie zależności: Dagger i Hilt
- 12** Warstwa domeny i wzorzec Use Case
- 13** Single Source of Truth i cache offline
- 14** Backend w chmurze: Firebase i Firestore

Drawer (szuflada nawigacyjna) to element interfejsu użytkownika, który działa jak wysuwane z boku (najczęściej z lewej krawędzi ekranu) menu.



Android Studio: Empty Activity Projekt Kotlin z interfejsem Jetpack Compose

Plik	Rola
gradle/libs.versions.toml	Wersje i aliasy bibliotek oraz pluginów
build.gradle.kts	Deklaracje pluginów projektu
app/build.gradle.kts	Pluginy i zależności modułu app

Dopisuj wpisy do istniejących sekcji.
Po zmianach wykonaj synchronizację projektu.

```
[versions]  
navigationCompose = "2.9.2"  
kotlinxSerialization = "1.7.3"
```

```
[libraries.androidx-navigation-compose]  
module = "androidx.navigation:navigation-compose"  
version.ref = "navigationCompose"
```

```
[libraries.kotlinx-serialization-json]  
module = "org.jetbrains.kotlinx:kotlinx-serialization-json"  
version.ref = "kotlinxSerialization"
```

W przykładzie: Navigation Compose 2.9.2.
Wersję pluginu Serialization dopasuj do Kotlin.

```
# gradle/libs.versions.toml
[plugins.kotlin-serialization]
id = "org.jetbrains.kotlin.plugin.serialization"
version.ref = "kotlin"

// build.gradle.kts: wewnątrz plugins
alias(libs.plugins.kotlin.serialization) apply false

// app/build.gradle.kts: wewnątrz plugins
alias(libs.plugins.kotlin.serialization)

// app/build.gradle.kts: wewnątrz dependencies
implementation(libs.androidx.navigation.compose)
implementation(libs.kotlinx.serialization.json)
```

version.ref = "kotlin" wskazuje wersję kompilatora.
Compose BOM nie dobiera wersji tych dwóch bibliotek.

Pojęcie	Odpowiedzialność
Trasa	Identyfikuje cel i określa argumenty
Ekran	Wyświetla interfejs użytkownika
Graf	Grupuje cele i określa cel początkowy
NavHost	Wiąże cele z UI i wyświetla bieżący ekran
NavController	Obsługuje przejścia i stos nawigacji

Graf zagnieżdżony grupuje ekrany.

Nie wymaga osobnego NavHost ani NavController.

```
import kotlinx.serialization.Serializable
```

```
@Serializable
```

```
data object Home
```

```
@Serializable
```

```
data class Details(val id: Int)
```

Home nie ma argumentów.

Details wymaga identyfikatora typu Int.

Typowane trasy są dostępne w Navigation Compose 2.
Do szczegółów przekazujemy ID, a nie cały obiekt danych.

@Composable

```
fun NavigationExample(modifier: Modifier = Modifier) {  
    val navController = rememberNavController()  
    NavHost(navController, Home, modifier) {  
        composable<Home> {  
            HomeScreen {  
                navController.navigate(Details(42))  
            }  
        }  
        composable<Details> { entry ->  
            val route = entry.toRoute<Details>()  
            DetailsScreen(route.id) {  
                navController.popBackStack()  
            }  
        }  
    }  
}
```

navigate(Details(42)) przekazuje argument.

toRoute<Details>() odczytuje go w ekranie docelowym.

@Serializable umożliwia Navigation zapisanie argumentów trasy i odtworzenie obiektu trasy przez toRoute()

@Composable

```
fun HomeScreen(onDetails: () -> Unit) {  
    Button(onClick = onDetails) {  
        Text("Szczegóły")  
    }  
}
```

@Composable

```
fun DetailsScreen(id: Int, onBack: () -> Unit) {  
    Column {  
        Text("ID: $id")  
        Button(onClick = onBack) { Text("Wstecz") }  
    }  
}
```

Ekran otrzymuje dane i zgłasza akcje.
Sposób przejścia określa kod konfigurujący graf.

Akcja	Ekran na stosie
Uruchomienie	Home
navigate(Details(42))	Home, Details(42)
navigate(Details(7))	Home, Details(42), Details(7)
popBackStack()	Home, Details(42)

Ostatni wpis odpowiada aktualnemu ekranowi.
Powrót usuwa wpis ze szczytu stosu.

```
navController.navigate(Home) {  
    popUpTo<Home> { inclusive = false }  
    launchSingleTop = true  
}
```

Przed	Po
Home, Details(42), Details(7)	Home

popUpTo usuwa wpisy ponad wskazanym celem.
inclusive = true usuwa także sam wskazany cel.

Jeden NavController

Szuflada i dolny pasek korzystają z tego samego grafu.

Szuflada

Ikona menu otwiera panel z profilem i ustawieniami.

Dolny pasek

Start i Koszyk przełączają główne obszary aplikacji.



Wspólny ekran aplikacji

```
val drawerState = rememberDrawerState(
    DrawerValue.Closed
)
val scope = rememberCoroutineScope()

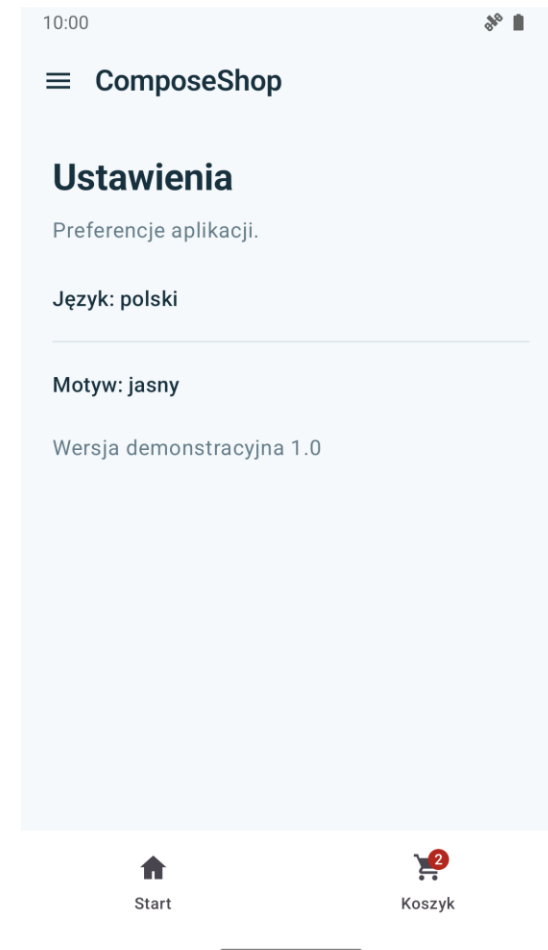
IconButton(onClick = {
    scope.launch {
        drawerState.open()
    }
}) {
    MenuIcon()
}
```

open() i close() są zawieszające.
Uruchamiamy je w scope.launch.



Animacja: otwarcie i wybór
Profilu

```
ModalNavigationDrawer(  
    drawerState = drawerState,  
    drawerContent = {  
        ModalDrawerSheet {  
            DrawerItems()  
        }  
    }  
) {  
    Scaffold(  
        topBar = { AppTopBar() },  
        bottomBar = { AppBottomBar() }  
    ) { padding ->  
        AppNavHost(  
            Modifier.padding(padding)  
        )  
    }  
}
```

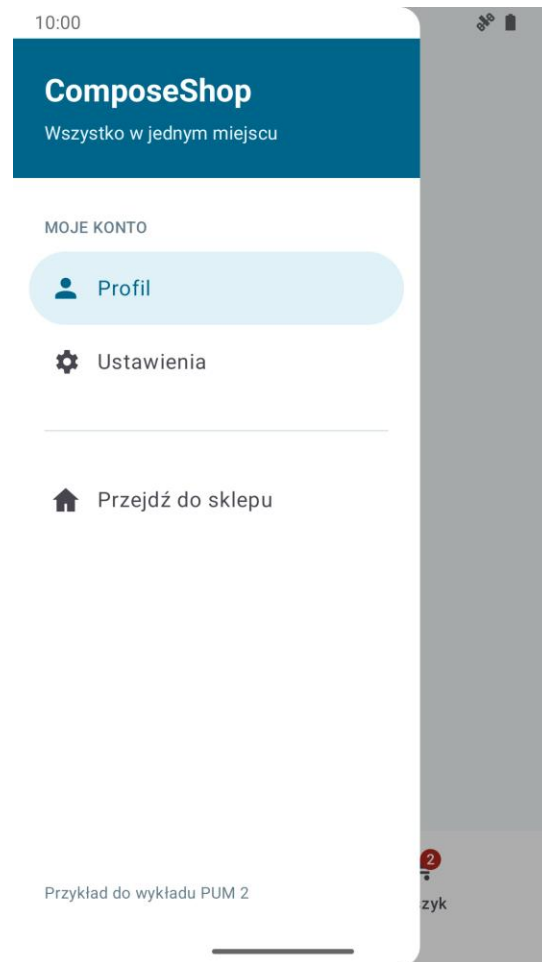


Scaffold: pasek górny, treść i
zakładki

```
val isProfile = current
    ?.hierarchy?.any {
        it.hasRoute<Profile>()
    } == true

NavigationDrawerItem(
    label = { Text("Profil") },
    selected = isProfile,
    onClick = {
        navController
            .openTopLevel(Profile)
            scope.launch {
                drawerState.close()
            }
    }
)
```

current pochodzi z aktualnego wpisu stosu. Zaznaczenie odpowiada trasie.



Profil zaznaczony w szufladzie

```
val isCart = current
    ?.hasRoute<Cart>() == true

NavigationBar {
    NavigationBarItem(
        selected = isCart,
        onClick = {
            navController
                .openTopLevel(Cart)
        },
        icon = { CartIcon() },
        label = { Text("Koszyk") }
    )
}
```

Start ma analogiczną pozycję.
CartIcon() wyświetla ikonę koszyka.



Koszyk jako aktywna zakładka

```
inline fun <reified T : Any>
NavController.openTopLevel(route: T) {
    val start = graph
        .findStartDestination().id
    navigate(route) {
        popUpTo(start) {
            saveState = true
        }
        launchSingleTop = true
        restoreState = true
    }
}
```

saveState i restoreState: stan zakładek.
launchSingleTop: bez nowej kopii celu,
gdy jest już na szczycie stosu.

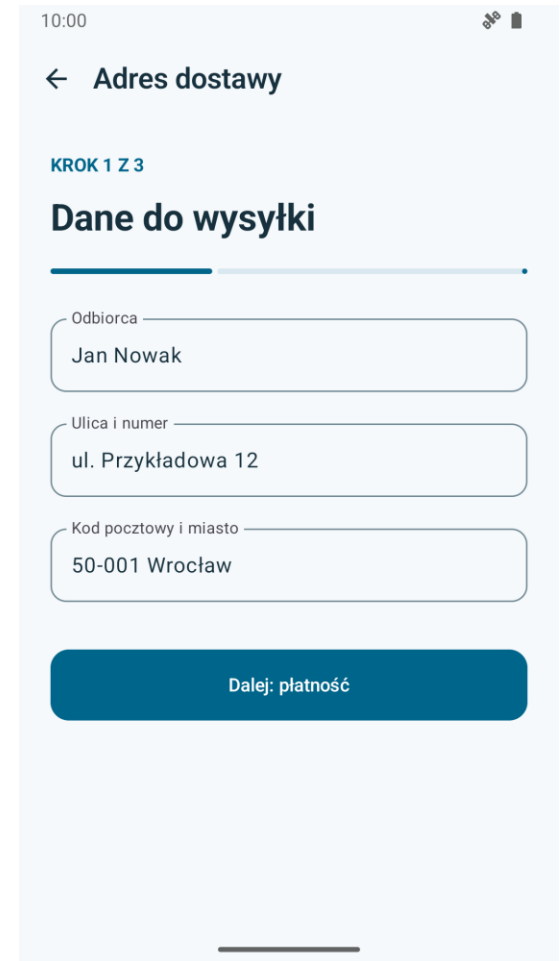


Powrót do zakładki Start

```
@Serializable data object Checkout  
@Serializable data object Address  
@Serializable data object Payment  
@Serializable data object Summary
```

```
navigation<Checkout>(  
    startDestination = Address  
) {  
    composable<Address> {  
        AddressScreen()  
    }  
    composable<Payment> {  
        PaymentScreen()  
    }  
    composable<Summary> {  
        SummaryScreen()  
    }  
}
```

Adres, płatność i podsumowanie
należą do jednego grafu Checkout.



Checkout zaczyna się od adresu

Wejście z koszyka

```
navController.navigate(Checkout)
```

Zakończenie zamówienia

```
navController.navigate(Home) {  
    popUpTo<Checkout> {  
        inclusive = true  
    }  
    launchSingleTop = true  
}
```

Wstecz nie otworzy już kroków zakupu.
Wcześniejsza historia może pozostać.



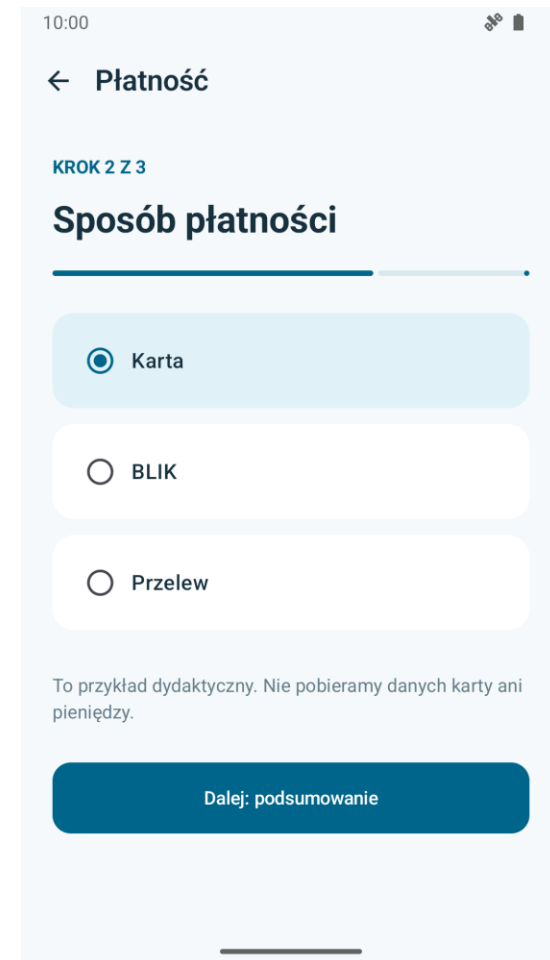
Po potwierdzeniu wraca ekran
Start

```
val inCheckout = current
    ?.hierarchy?.any {
        it.hasRoute<Checkout>()
    } == true

LaunchedEffect(inCheckout) {
    if (inCheckout) drawerState.close()
}

// ModalNavigationDrawer:
gesturesEnabled = !inCheckout

// Scaffold:
bottomBar = {
    if (!inCheckout) AppBottomBar()
}
```



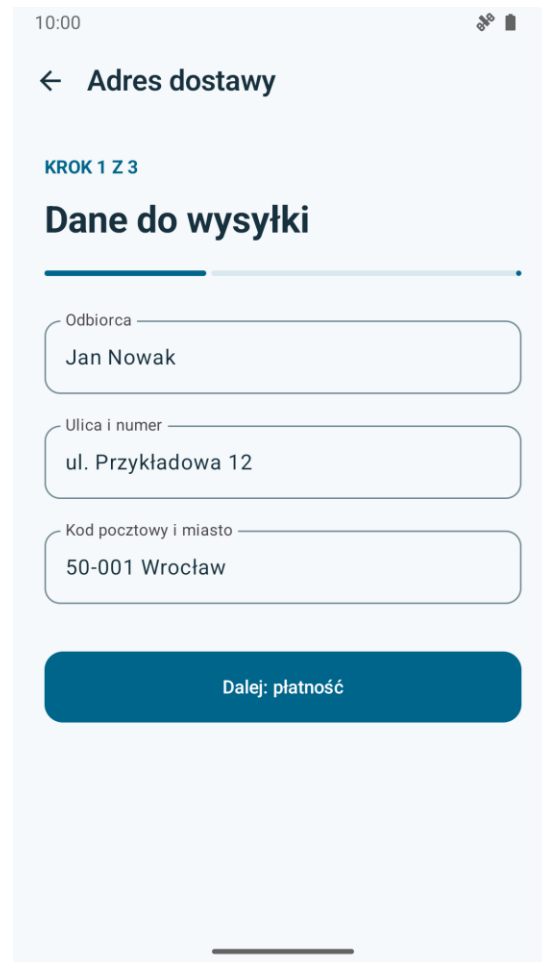
Płatność bez szuflady i dolnego paska

```
// Stan ponad NavHost
var address by rememberSaveable {
    mutableStateOf("")
}
var payment by rememberSaveable {
    mutableStateOf("Karta")
}
```

Wartości i callbacki

Ekrany odczytują wspólny stan
i zgłaszają jego zmiany.

Powrót do poprzedniego kroku
nie kasuje danych. Po potwierdzeniu
zamówienia czyścimy formularz.



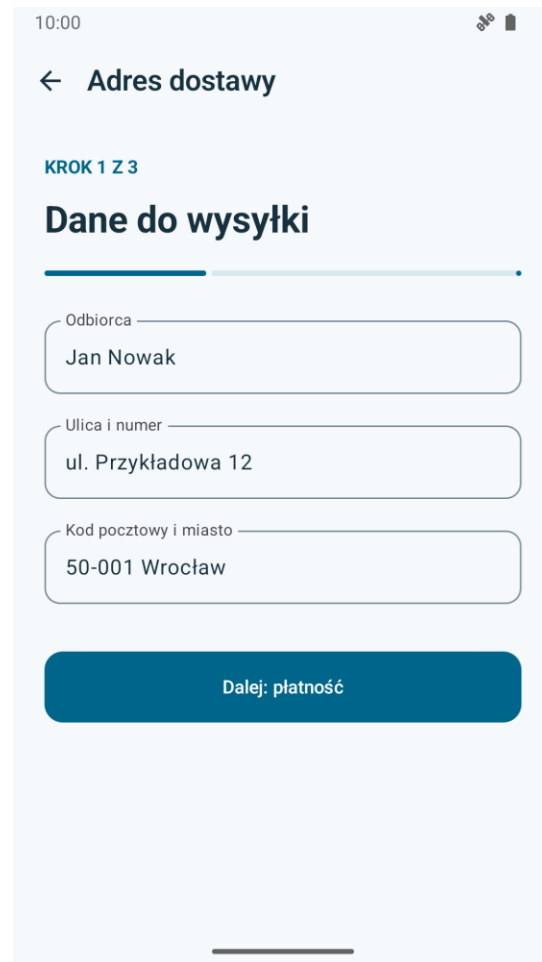
Adres zachowany po powrocie
z płatności

```
// Stan ponad NavHost
var address by rememberSaveable {
    mutableStateOf("")
}
var payment by rememberSaveable {
    mutableStateOf("Karta")
}
```

Wartości i callbacki

Ekrany odczytują wspólny stan
i zgłaszają jego zmiany.

Powrót do poprzedniego kroku
nie kasuje danych. Po potwierdzeniu
zamówienia czyścimy formularz.



Adres zachowany po powrocie
z płatności